Fundamentals of AI and ML Project

Snake Game AI

Submitted by :

Name: Pari Porwal

Registration Number: 25BAI10001

Branch: B.Tech (AI & ML)

Course Code: CSA2001

Course Title: Fundamentals in AI and M

Acknowledgement

I would also like to thank my instructor for their guidance, support, and feedback during the creation of this project. With their guidance, this tutoring became a great support for me to really grasp the ties behind Artificial Intelligence and the NEAT model in particular.

I am also grateful to my institution for the resources they provided and a supportive environment. Thanks to my peers for their valuable discussions and motivation.

Introduction

Artificial Intelligence (AI) is evolving at a rapid pace in every industry and the gaming space has not remained untouched. Traditional agents use prescribed & predetermined rules but AI-based agent learn and adapt over the period.

The Snake Game is a relatively simple but hard environment in which an agent has to:

- Avoid collisions
- Navigate efficiently
- More food collected = higher score

Because of this, we will use the NEAT algorithm for evolving neural networks that control the snake. Rather than hard-coding behavior, the AI discovers optimal strategies through experience

Objective

The main objectives of this project are:

- To design and develop a Snake Game using Python
- To implement an AI agent using NEAT
- To train the AI over multiple generations
- To analyze performance using graphical representation
- To understand evolutionary algorithms in real-world scenarios

Tools & Technologies

- Python: Main programming language
- Pygame: Used for game development
- NEAT-Python: Implements NEAT algorithm
- Matplotlib: Used for plotting training graphs

System Architecture

The system consists of four main components:

1. Game Environment

The game environment, written with Python and the Pygame library, handles registering every aspect of the game — including the snake itself, its food for sustenance trying to emerge onto a 640 × 480 pixel screen bounded by two walls. It dynamically adjusts the game state in response to the agent's actions and delivers feedback as rewards or penalties, serving as mechanisms for assessing performance.

2. AI Agent (Neural Network)

The next state is the snake controlled by an AI agent (the neural network). It takes inputs like the position of food, nearby obstacles and current direction.

3. Evolution Engine (NEAT Algorithm)

The NEAT algorithm trains the neural networks through many generations. It achieves enhanced performance through the use of genetic operations such as mutation and crossover, in addition to optimizing both the network structure and weights.

4. Agent–Environment Interaction

It is based on a perpetual cycle of perception, action and feedback. On the basis of fitness received by taking action in the environment, agent is learning and updating parameter from this feedback.

Working of the Game

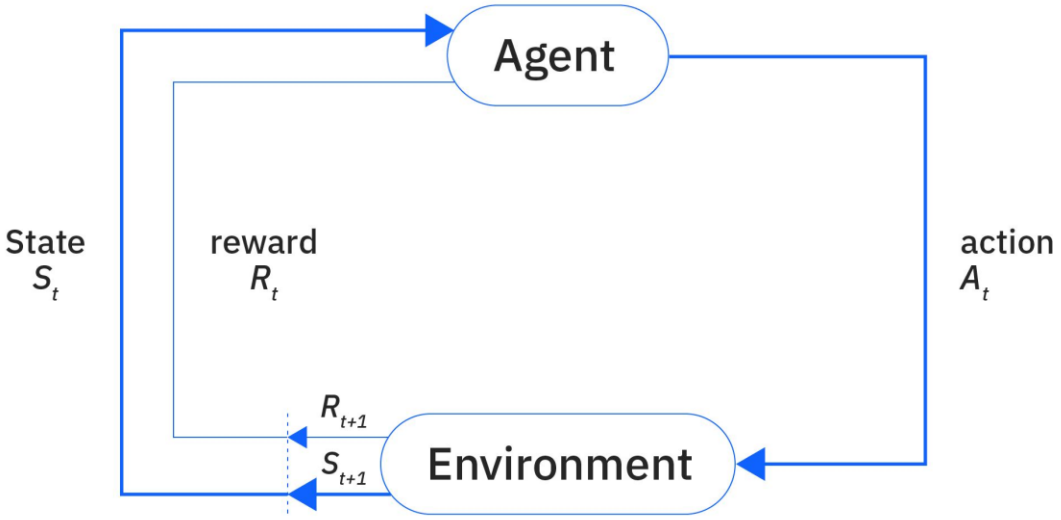
The snake moves in a grid environment. The rules include:

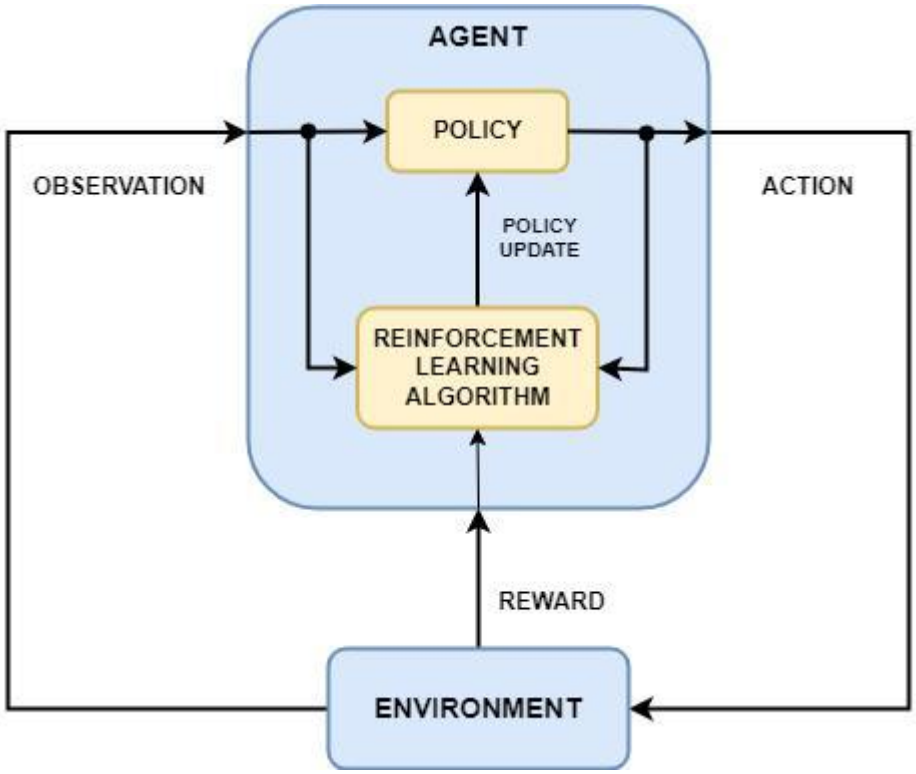
- The snake grows when it eats food
- Game ends on collision with wall or itself
- Score increases with each food item

The AI controls the snake by making decisions based on its surroundings.

Agent–Environment Concept

The agent and environment concept is fundamental in Artificial Intelligence. Agents are entities that observe their environment via inputs (for example, distance to food or obstacles) and perform actions to achieve a goal. In this project, the environment is made up of the game grid, food and obstacles while the snake serves as an agent. The agent observes the state of its environment and uses a neural network to pick its next move.The environment responds to these actions by updating the game state. This continuous interaction enables learning, especially when combined with evolutionary algorithms like NEAT, which optimize the agent's behavior over time.





The figure represents the interaction between the agent and the environment. The agent (snake) receives inputs about the environment, like where the food / walls are and its own body. The neural network takes these inputs and determines an action to follow (one of four possible actions: up, down, left or right).

The environment reacts to that action, updating the game state accordingly — for example by moving a snake or triggering a lost game state.This continuous loop of **perception → decision → action → feedback** allows the agent to learn and improve over time using the NEAT algorithm.

NEAT Algorithm Explanation

The NEAT (NeuroEvolution of Augmenting Topologies) algorithm is an advanced evolutionary technique used to train neural networks by evolving both their structure and connection weights. Unlike traditional neural networks, where the architecture is fixed, NEAT starts with simple networks and gradually increases their complexity over generations.

A population of small neural networks is instantiated at first, with rudimentary connections from inputs to outputs. At training, each of the network is evaluated by a fitness function with its performance on its game. Genetic algorithms are used to determine which of the networks performs best, so it can reproduce and form the next generation.

NEAT then uses genetic operators like mutation and crossover. Mutation changes connection weights, adds nodes or connections on the network enabling it to expand and adapting. Crossover combines features of two parent networks to produce offspring with potentially better performance.

An important feature of NEAT is speciation, where similar networks are grouped into species. This protects innovative structures from being eliminated too early and maintains diversity in the population. Over time, NEAT evolves increasingly efficient neural networks, enabling the agent to learn complex behaviors and improve its performance in the Snake Game.

Training Process

Using NEAT each agent must play Snake Game for several generations. First, neural networks with random weights and simple structure are generated in a population. Each network is controlling an independent snake, which plays the game separately.

At every generation, each agent interacts with the environment by seeing inputs: position of food, barriers, and current orientation. The neural network uses these inputs to determine how the snake should move. Each agent is assigned a fitness score, depending on its performance, which in most cases depends on survival time and quantity of food gathered.

Once all agents have been evaluated, the NEAT algorithm selects superior networks and performs genetic operations (e.g., mutation, crossover) to produce a fresh population. With each generation, the worst performing agents are removed and the most successful agents develop further. This training loop repeats until the AI successfully plays the game proficiently, resulting in decisions that get better and scores that increase over time.

Fitness Function

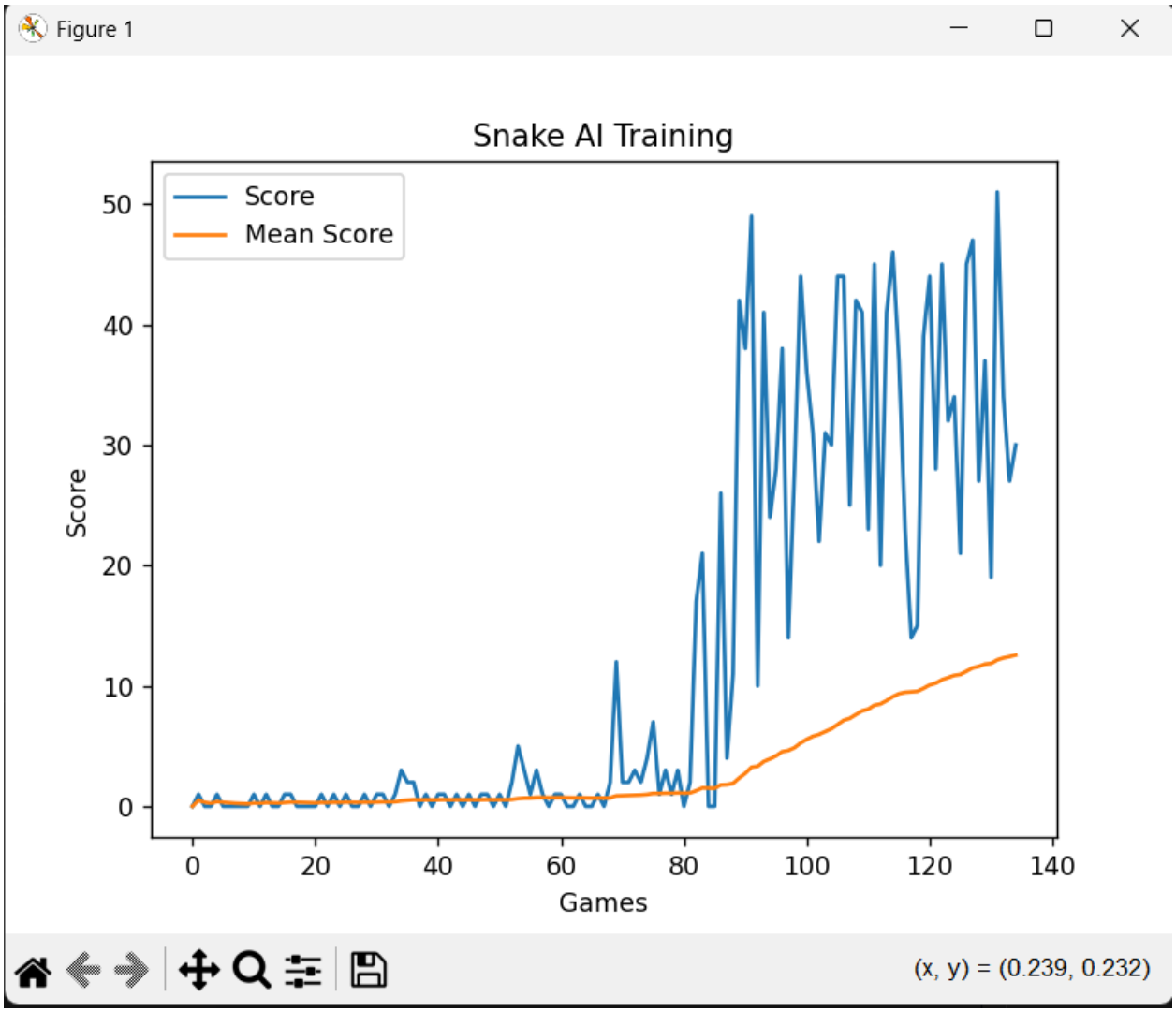
Fitness Function: This is an essential part of the training process which shines a light on how well each AI agent did in playing the Snake Game. It returns a numerical score that denotes how well the agent performed, which is used to help the NEAT algorithm select neural networks for replication that performed better in aggregate during its lifetime.

In the current project, we use fitness value calculated based on multiple factors. The longer the agent stays alive, and the more food he successfully collects (i.e. good performance), the higher reward it gets. For example, a higher reward is given when the snake eats food, while a smaller reward may be given for staying alive for a longer duration. On the other hand, penalties may be applied if the snake collides with walls or itself, or if it moves inefficiently.

The fitness function should be well thought out so that the agent is not learning random strategies but meaningful ones. It promotes behaviors like optimizing your path, not colliding with anything and even moving towards a given target. The fitness function is critical in creating the best outcome by constantly measuring success and leading the agents through successful actions.

Training Results

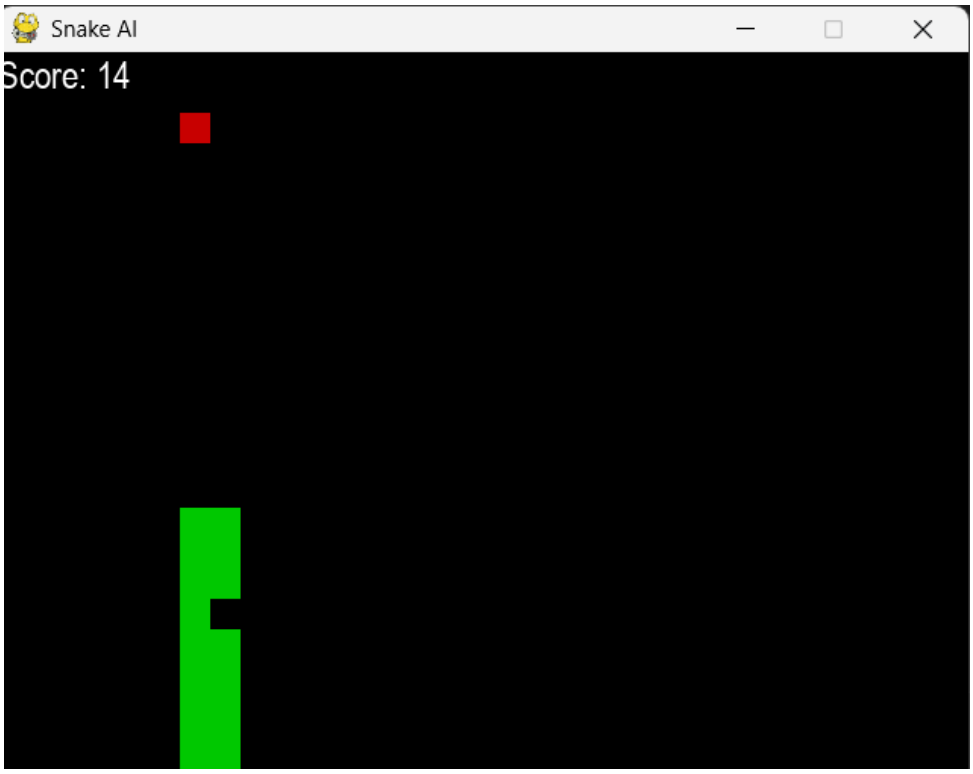
Learning Graph :

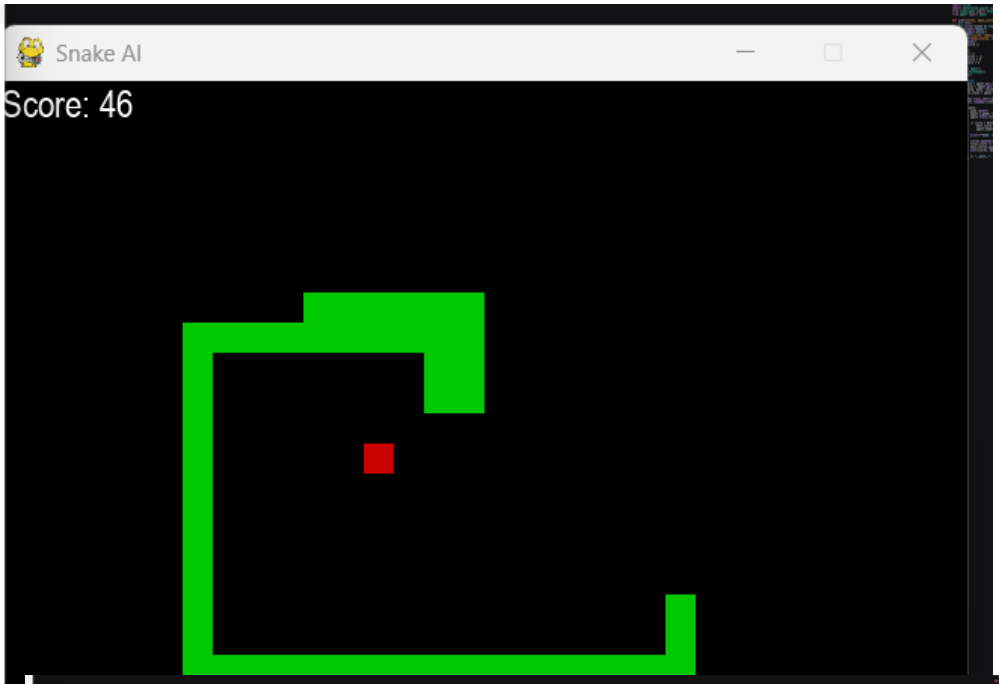


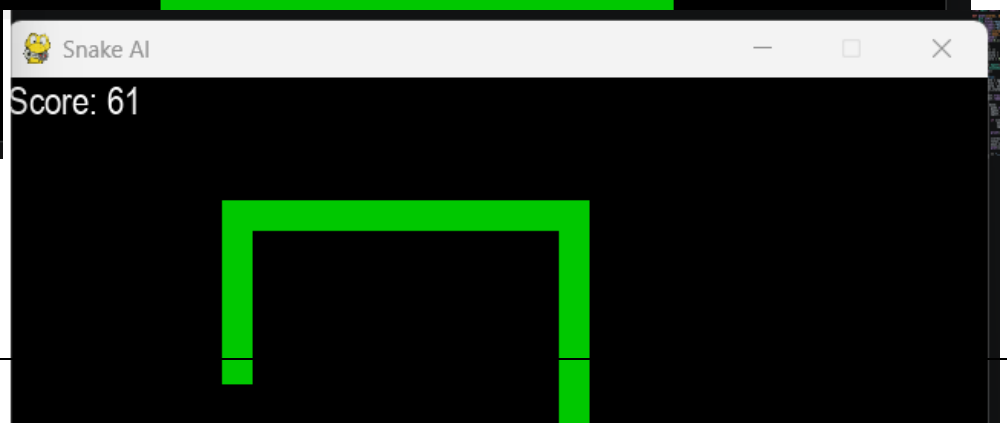
Explain:

- X-axis: Generations
- Y-axis: Fitness score
- Shows improvement trend

Game Output


Snake AI
Score: 14


Snake AI
Score: 46


Snake AI
Score: 61

Terminal output :

```
Game: 1 | Score: 0 | Best: 0
Game: 2 | Score: 1 | Best: 1
Game: 3 | Score: 0 | Best: 1
Game: 4 | Score: 0 | Best: 1
Game: 5 | Score: 1 | Best: 1
Game: 6 | Score: 0 | Best: 1
Game: 7 | Score: 0 | Best: 1
Game: 8 | Score: 0 | Best: 1
Game: 9 | Score: 0 | Best: 1
Game: 10 | Score: 0 | Best: 1
Game: 11 | Score: 1 | Best: 1
```

Game: 12 | Score: 0 | Best: 1

Game: 13 | Score: 1 | Best: 1

Game: 14 | Score: 0 | Best: 1

Game: 15 | Score: 0 | Best: 1

Game: 16 | Score: 1 | Best: 1

Game: 17 | Score: 1 | Best: 1

Game: 18 | Score: 0 | Best: 1

Game: 19 | Score: 0 | Best: 1

Game: 20 | Score: 0 | Best: 1

Game: 21 | Score: 0 | Best: 1

Game: 22 | Score: 1 | Best: 1

Game: 23 | Score: 0 | Best: 1

Game: 24 | Score: 1 | Best: 1

Game: 25 | Score: 0 | Best: 1

Game: 26 | Score: 1 | Best: 1

Game: 27 | Score: 0 | Best: 1

Game: 28 | Score: 0 | Best: 1

Game: 29 | Score: 1 | Best: 1

Game: 30 | Score: 0 | Best: 1

Game: 31 | Score: 1 | Best: 1

Game: 32 | Score: 1 | Best: 1

Game: 33 | Score: 0 | Best: 1

Game: 34 | Score: 1 | Best: 1

Game: 35 | Score: 3 | Best: 3

Game: 36 | Score: 2 | Best: 3

Game: 37 | Score: 2 | Best: 3

Game: 38 | Score: 0 | Best: 3

Game: 39 | Score: 1 | Best: 3

Game: 40 | Score: 0 | Best: 3

Game: 41 | Score: 1 | Best: 3

Game: 42 | Score: 1 | Best: 3

Game: 43 | Score: 0 | Best: 3

Game: 44 | Score: 1 | Best: 3

Game: 45 | Score: 0 | Best: 3

Game: 46 | Score: 1 | Best: 3

Game: 47 | Score: 0 | Best: 3

Game: 48 | Score: 1 | Best: 3

Game: 49 | Score: 1 | Best: 3

Game: 50 | Score: 0 | Best: 3

Game: 51 | Score: 1 | Best: 3

Game: 52 | Score: 0 | Best: 3

Game: 53 | Score: 2 | Best: 3

Game: 54 | Score: 5 | Best: 5

Game: 55 | Score: 3 | Best: 5

Game: 56 | Score: 1 | Best: 5

Game: 57 | Score: 3 | Best: 5

Game: 58 | Score: 1 | Best: 5

Game: 59 | Score: 0 | Best: 5

Game: 60 | Score: 1 | Best: 5

Game: 61 | Score: 1 | Best: 5

Game: 62 | Score: 0 | Best: 5

Game: 63 | Score: 0 | Best: 5

Game: 64 | Score: 1 | Best: 5

Game: 65 | Score: 0 | Best: 5

Game: 66 | Score: 0 | Best: 5

Game: 67 | Score: 1 | Best: 5

Game: 68 | Score: 0 | Best: 5

Game: 69 | Score: 2 | Best: 5

Game: 70 | Score: 12 | Best: 12

Game: 71 | Score: 2 | Best: 12

Game: 72 | Score: 2 | Best: 12

Game: 73 | Score: 3 | Best: 12

Game: 74 | Score: 2 | Best: 12

Game: 75 | Score: 4 | Best: 12

Game: 76 | Score: 7 | Best: 12

Game: 77 | Score: 1 | Best: 12

Game: 78 | Score: 3 | Best: 12

Game: 79 | Score: 1 | Best: 12

Game: 80 | Score: 3 | Best: 12

Game: 81 | Score: 0 | Best: 12

Game: 82 | Score: 2 | Best: 12

Game: 83 | Score: 17 | Best: 17

Game: 84 | Score: 21 | Best: 21

Game: 85 | Score: 0 | Best: 21

Game: 86 | Score: 0 | Best: 21

Game: 87 | Score: 26 | Best: 26

Game: 88 | Score: 4 | Best: 26

Game: 89 | Score: 11 | Best: 26

Game: 90 | Score: 42 | Best: 42

Game: 91 | Score: 38 | Best: 42

Game: 92 | Score: 49 | Best: 49

Game: 93 | Score: 10 | Best: 49

Game: 94 | Score: 41 | Best: 49

Game: 95 | Score: 24 | Best: 49

Game: 96 | Score: 28 | Best: 49

Game: 97 | Score: 38 | Best: 49

Game: 98 | Score: 14 | Best: 49

Game: 99 | Score: 28 | Best: 49

Game: 100 | Score: 44 | Best: 49

Game: 101 | Score: 36 | Best: 49

Game: 102 | Score: 31 | Best: 49

Game: 103 | Score: 22 | Best: 49

Game: 104 | Score: 31 | Best: 49

Game: 105 | Score: 30 | Best: 49

Game: 106 | Score: 44 | Best: 49

Game: 107 | Score: 44 | Best: 49

Game: 108 | Score: 25 | Best: 49

Game: 109 | Score: 42 | Best: 49

Game: 110 | Score: 41 | Best: 49

Game: 111 | Score: 23 | Best: 49

Game: 112 | Score: 45 | Best: 49

Game: 113 | Score: 20 | Best: 49

Game: 114 | Score: 41 | Best: 49

Game: 115 | Score: 46 | Best: 49

Game: 116 | Score: 37 | Best: 49

Game: 117 | Score: 23 | Best: 49

Game: 118 | Score: 14 | Best: 49

Game: 119 | Score: 15 | Best: 49

Game: 120 | Score: 39 | Best: 49

Game: 121 | Score: 44 | Best: 49

Game: 122 | Score: 28 | Best: 49

Game: 123 | Score: 45 | Best: 49

Game: 124 | Score: 32 | Best: 49

Game: 125 | Score: 34 | Best: 49

Game: 126 | Score: 21 | Best: 49

Game: 127 | Score: 45 | Best: 49

Game: 128 | Score: 47 | Best: 49

Game: 129 | Score: 27 | Best: 49

Game: 130 | Score: 37 | Best: 49

Game: 131 | Score: 19 | Best: 49

Game: 132 | Score: 51 | Best: 51

Game: 133 | Score: 34 | Best: 51

Game: 134 | Score: 27 | Best: 51

Game: 135 | Score: 30 | Best: 51

Game: 136 | Score: 16 | Best: 51

Analysis

The AI agent shows clear progression throughout generations as it performs better. It demonstrates the success of the evolutionary learning method. In the beginning, because it knows nothing of the past, the agent just moves randomly and will "sometimes" hit walls or itself. Across generations, the agent slowly learns things like moving towards food, and away from obstacles.This improvement is driven by the interaction between the **agent and its environment**, where the snake (agent) continuously observes its surroundings (environment) and takes actions based on neural network outputs. The NEAT algorithm plays an important role by evolving both the structure and weights of the neural networks, leading to more efficient strategies to build over time.

Future Scope

You can improve upon this project with more complicated environments and sophisticated learning processes. Adding obstacles, introducing an agent to interact with others are all valid ways to make the agent environment more complex. Moreover, deep learning techniques like reinforcement learning could be applied to enhance efficiency on decision making in the same way as NEAT algorithm does. We can further improve the current agent by fine-tuning the fitness function and input parameters to attain even faster and more stable learning.Moreover , this approach can be extended to other real-world problems, such as robotics and autonomous systems, where intelligent agents must interact with their environment to achieve specific goals.

Conclusion

An overview of the project: This project is an excellent demonstration of using Artificial Intelligence in Game Development leveraging NEAT (Neuro Evolution of Augmenting Topologies). The Snake Game serves as a useful environment to demonstrate how an intelligent agent can learn and enhance its performance without being explicitly programmed. As the agent will interact continually with its environment, the AI segment alerts from random motion to more optimized strategies such as proper navigation and distance avoidance. These evolutionary approaches drive the neural network to adjust its shape and weights, leading to better decision making Despite some drawbacks like the time it takes to train or needing careful tuning of its parameters, this model offers insight into evolution as a learning process. In conclusion, the project is a great demonstration of the capabilities of AI in providing solutions to complex problems and gives me much directions for more projects building on intelligent agent systen and machine learning.

References

- NEAT-Python Documentation
  https://neat-python.readthedocs.io/
- Pygame Documentation
  https://www.pygame.org/docs/
- Python Official Documentation
  https://docs.python.org/3/
- Stanley, K. O., & Miikkulainen, R. (2002).
  Evolving Neural Networks through Augmenting Topologies (NEAT)
  Journal of Evolutionary Computation
- Matplotlib Documentation
  https://matplotlib.org/stable/contents.html
- Online tutorials and resources related to AI-based Snake Game development